

Challenge 9: Neural Flappy Bird

Difficulty: Expert

Points: 300 + Judge Bonus (+ 200 / 150 / 100)

Category: Combined, FPGA + ESP32

Description

Build an embedded Flappy Bird system that starts as a manually controlled game, evolves into an ESP32 neural-network training simulation, and finishes with hardware neural-network inference on the FPGA.

The ESP32 renders the game on the OLED display. The FPGA acts first as the game controller and difficulty controller, then as the hardware inference engine for the trained neural network.

Milestone 1: Manual FPGA-Controlled Game (80 points)

Goal

Verify that FPGA-to-ESP32 communication works, and that FPGA inputs can control a live game running on the ESP32.

Requirements

- ESP32 renders a playable Flappy Bird-style game on the OLED display.
- FPGA reads KEY[0] as the flap button.
- On every valid button press, FPGA sends a flap command to the ESP32 over UART.
- ESP32 receives the flap command and moves the bird upward.
- If the bird hits the floor, ceiling, or obstacle, the game ends.
- Pressing KEY[0] after game over restarts the game.
- FPGA reads SW[3:0] as a difficulty value from 0 to 15 (by the binary value).
- FPGA displays the current difficulty value on the 7-segment display.
- FPGA sends the difficulty value to the ESP32 over UART.
- Higher difficulty increases obstacle speed.
- Higher difficulty reduces the obstacle gap/free space.

Milestone 2: ESP32 Neural-Network Training (100 points)

Goal

Train a simple neural network directly on the ESP32 and show visible improvement over generations.

Requirements

- ESP32 replaces manual control with a training simulation.
- ESP32 simulates a population of at least 30 birds in parallel.
- Each bird has its own neural network.
- Each neural network receives game-state inputs such as:
 - Bird height
 - Bird velocity
 - Distance to the next obstacle
 - Distance to the obstacle gap
- The neural network has:
 - One hidden layer with 4 neurons
 - One output neuron that decides whether to flap
- All birds run on the same deterministic obstacle course within each generation.
- Fitness is based on survival time, progress, or score.
- After each generation, the ESP32 keeps the top-performing birds.
- The next generation is created from the best birds.
- New birds are generated by copying selected parents (tip - add small random weight mutations. Occasional larger mutations may be used to improve exploration).
- OLED shows:
 - Live population
 - Generation number
 - Alive count
 - Best score
- The system should show visible improvement over generations.

Milestone 3: FPGA Neural-Network Inference (120 points)

Goal

Use the FPGA not only as an input and configuration device, but also as a hardware accelerator that runs the trained neural network inference.

Requirements

- One FPGA switch selects the operating mode:
 - ESP32 training mode
 - FPGA inference mode
- In training mode:
 - The ESP32 trains the neural networks.
 - The OLED displays training progress.
- After training:
 - The best neural-network weights are transferred or copied to the FPGA.
- In inference mode:
 - The ESP32 sends the current game-state inputs to the FPGA.
 - The FPGA runs the trained neural-network inference.
 - The FPGA returns the flap/no-flap decision to the ESP32.
 - The ESP32 uses the FPGA decision to control the bird in the live game.
- OLED clearly indicates the current operating mode.
- The FPGA inference result must visibly control the game without manual button input.

Rules

- The game must run live on the ESP32 and OLED display.
- The FPGA must be part of the control path in Milestone 1.
- FPGA switches must affect game difficulty live.
- Neural-network training must run directly on the ESP32, not on a PC.
- For Milestone 3, the final flap/no-flap decision must be computed on the FPGA.

Judge Bonus

Additional bonus points may be awarded to the best complete systems.

Place	Bonus
-------	-------

1st place	+250
-----------	------

2nd place	+200
-----------	------

3rd place	+150
-----------	------

The judge bonus is based on:

- Gameplay quality
 - Training quality
 - Visible learning improvement
 - FPGA inference implementation
 - System robustness
 - Demo clarity
 - Creativity and polish
-